

Code-Layout, Readability and Reusability

Date	15 July 2026
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

This document evaluates the quality of the Rising Waters codebase in terms of its structure, readability, and reusability.

Code Layout Checklist

S. No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation used consistently across app.py, train.py, and core/ modules
2	Proper File Structure	Files and folders are logically organized	Yes	Organized into core/, templates/, static/, models/, dataset/, notebooks/, tests/
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. cloud_cover, annual_rainfall, jan_feb, mar_may, jun_sep, FEATURE_COLUMNS
4	Function / Method Names	Functions are descriptively named	Yes	e.g. predict(), load_real_dataset(), build_synthetic_dataset(), validate_field()
5	Code Comments	Inline and block comments explain logic	Yes	Comments explain preprocessing steps, model loading, and validation bounds
6	Modular Design	Code is split into reusable functions/modules	Yes	Validation, prediction, and config logic separated from Flask routes
7	No Redundant Code	Duplicate or unused code is removed	Partial	Some overlap between train.py's real-data and synthetic-data paths could be refactored further
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except around form parsing; invalid input returns to the form with a clear message instead of crashing

Reusable Components / Modules

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	core/validation.py	Python	Called from app.py's /predict route for all incoming form data	High

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
2	core/predictor.py	Python, joblib, XGBoost	Called from app.py to scale inputs and run inference	High
3	static/css/style.css design tokens	CSS custom properties	Shared across home.html, index.html, chance.html, nochance.html	High
4	draw_process() / draw_datastore() helpers	Python, reportlab	Reused across every box in the Data Flow Diagram generator	Medium
5	build_synthetic_dataset()	Python, NumPy, Pandas	Fallback path in train.py whenever raw_data/ has no rainfall file	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of concerns across core/, templates/, static/, and models/
Readability	5	Descriptive names, consistent style, and comments that explain intent, not just mechanics
Reusability	4	Core logic is decoupled from Flask routing, though some cross-script duplication remains in train.py
Documentation / Comments	4	Good in-code comments plus a full README covering setup, architecture, and run instructions
Overall Score	4.5	Solid, maintainable structure for a solo project; minor refactor opportunity in train.py